1. Semantic Understanding and Command Following (Text Version)

1. Course Content

1. Learn the fundamental principles by which large models understand natural-language user commands and orchestrate complex actions;
2. Through terminal text interaction (`text_chat`), issue a sequence of control commands to the robot to verify the large model's semantic parsing and command-following capabilities for motion distance, turning angle, speed, and composite entertainment actions;
3. Analyze the underlying collaboration logic between the core source code `model_service.py` and the action server `action_service_*.py` .

---

2. Preparation

2.1 Content Description

> This lesson applies to controllers such as Raspberry Pi 5, Jetson Orin NANO, and D-Robotics RDK X5:
>
> - **Raspberry Pi 5**: You must first enter the Docker container before running ROS 2 related commands. Refer to [3.Enter Docker (For Raspberry Pi)] in the [Robot Configuration and Operation Guide];
> - **Orin / RDK X5**: Enter commands directly in the onboard terminal; there is no need to enter Docker.

2.2 Start the Local Dify Large-Model Environment

Enter the following command in the terminal to start the Dify service:

```
sh ~/bringup_dify.sh
```



---

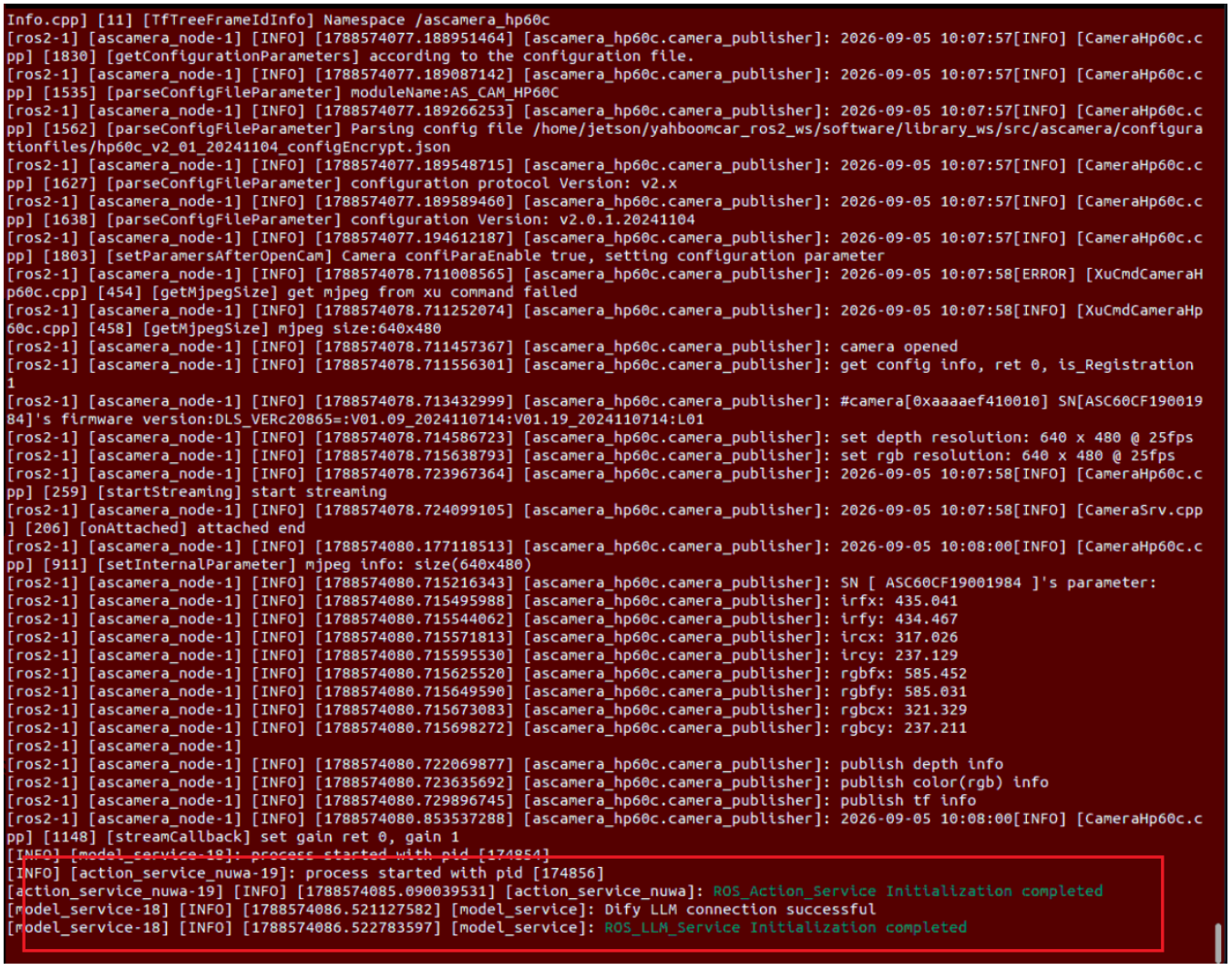
3. Running Examples

3.1 Start the Embodied Intelligence Master Control Node (Text Interaction Mode)

Open a terminal on the onboard computer and enter the following command to start the large-model embodied control service (specifying `text_chat_mode:=True`):

```
ros2 launch multi_brains llm_agent_control.launch.py text_chat_mode:=True
```

After the node initializes, the terminal prints messages indicating that the model has loaded and the Action service is ready:

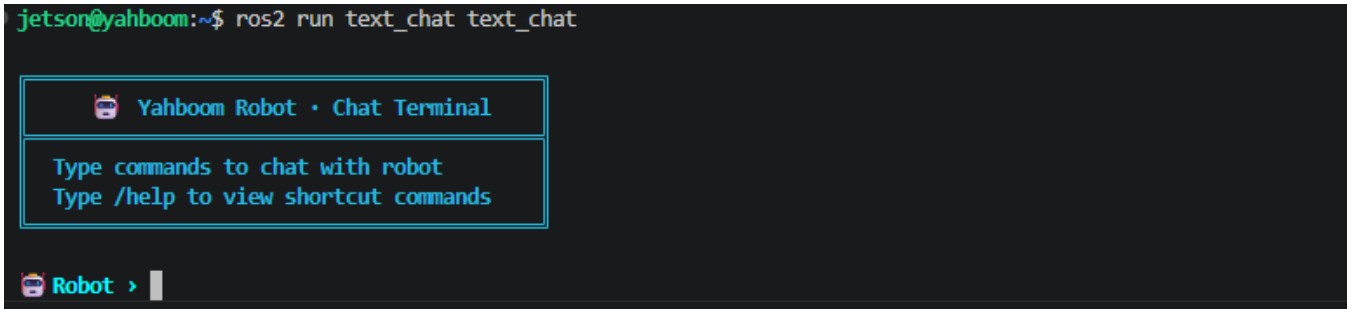


3.2 Start the Text Interaction Control Terminal

Use **remote SSH** to open a terminal for text interaction:

```
ros2 run text_chat text_chat
```

After startup, the terminal enters the real-time text interaction interface, waiting for the user to enter commands:



> **Tip**
>
> **Chinese Input Tip**:
>
> - It is recommended to connect from a computer using an SSH remote tool (such as MobaXterm or VSCode Terminal), so that Chinese can be typed seamlessly.

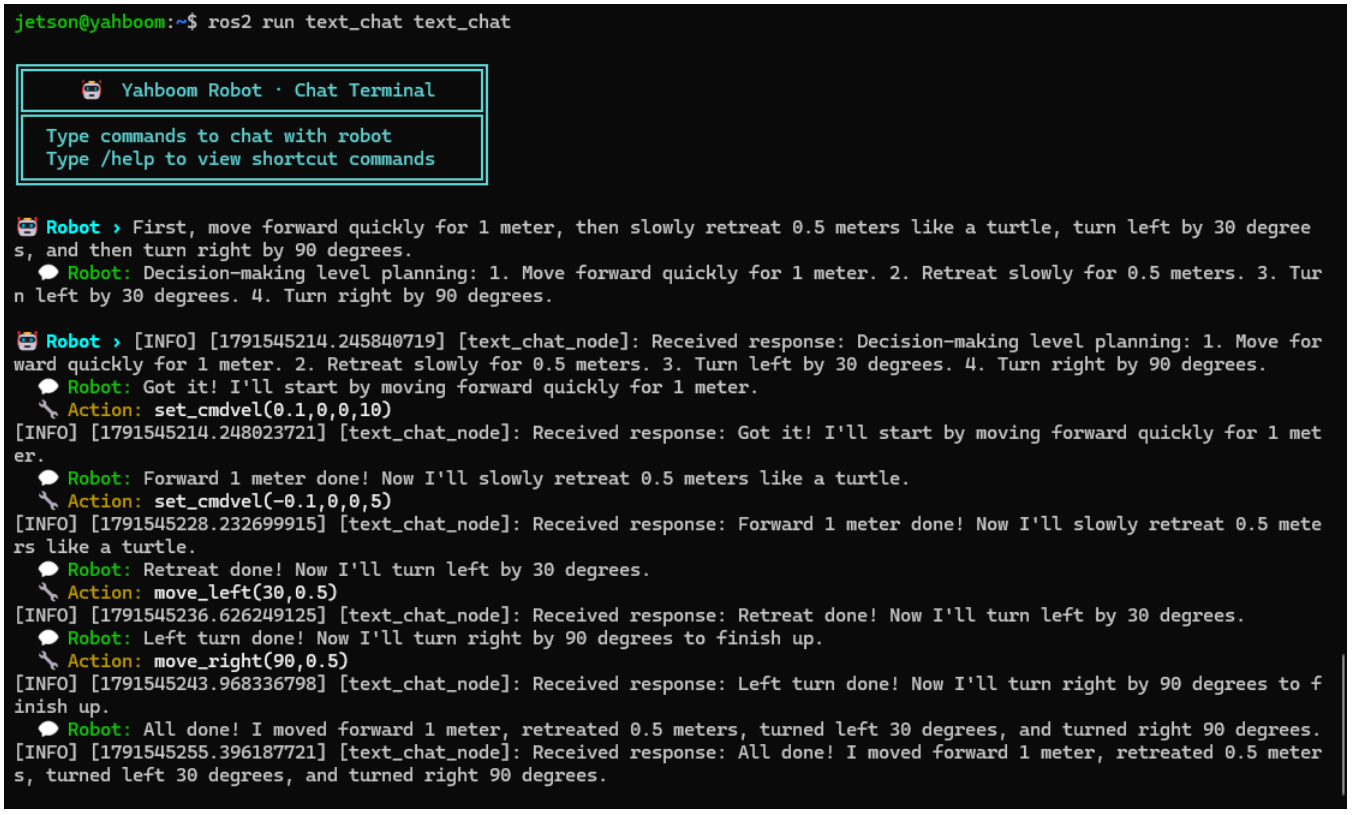
3.3 Classic Test Cases

In the text interaction terminal, enter the following classic commands one by one to test semantic understanding and actions:

Case 1: Chassis Motion Speed and Composite Displacement Control

Enter the test statement in the terminal: **(Note: Left and right turns require a large space; it is recommended to elevate the robot for testing or to ensure that there is sufficient space around it.)**

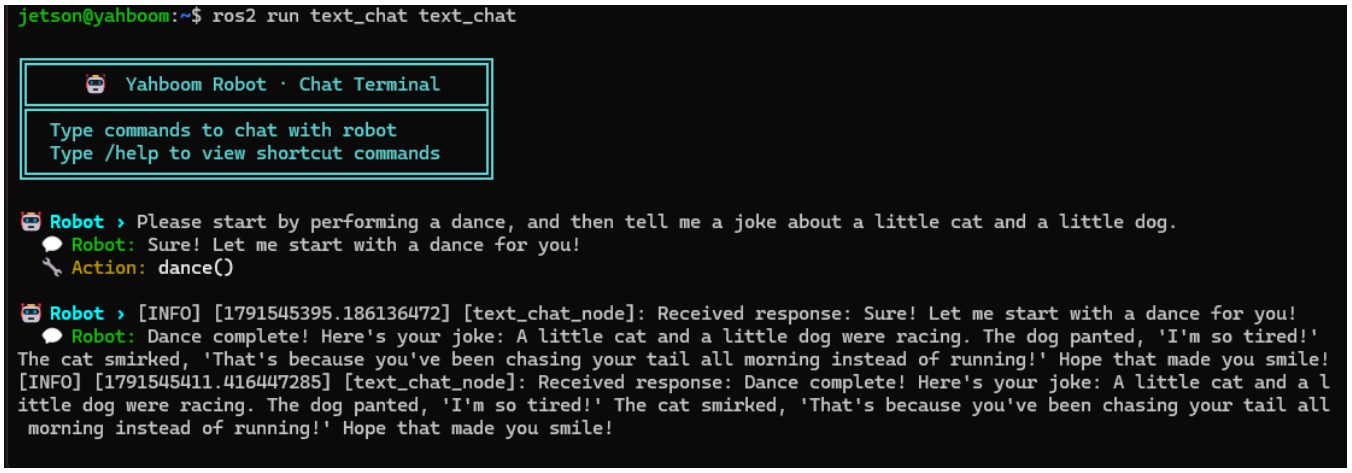
```
First, move forward quickly for 1 meter, then slowly retreat 0.5 meters like a turtle, turn left by 30 degrees, and then turn right by 90 degrees.
```



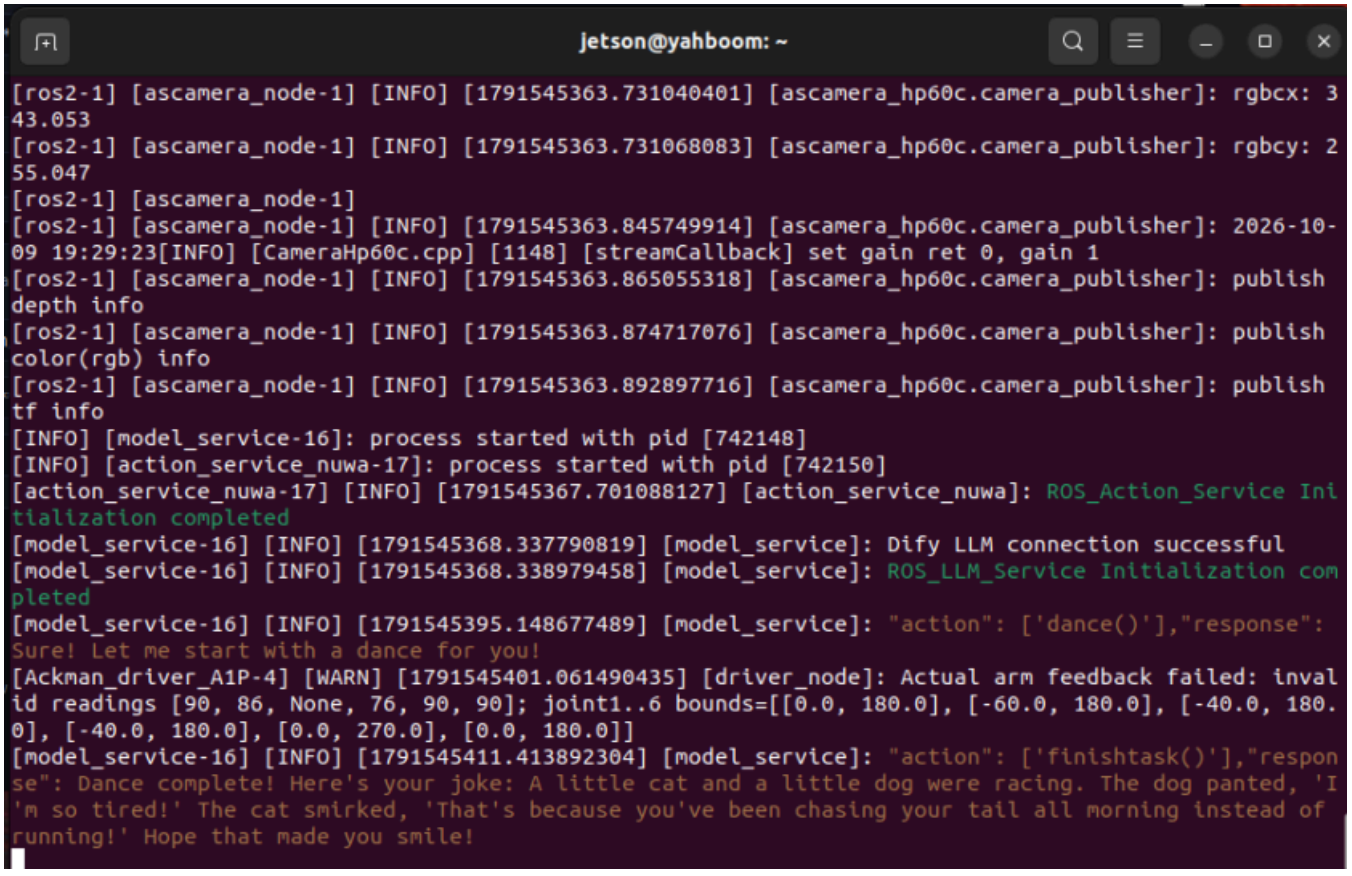
Case 2: Anthropomorphic Performance and Fun Interaction

Enter the test statement in the terminal:

```
Please start by performing a dance, and then tell me a joke about a little cat and a little dog.
```



Execution Process and Behavior



4. Core Source Code Analysis

Source package path:

`~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains`

4.1 Large-Model Decision Hub (model_service.py)

In `LargeModelService`, when the text chat mode `text_chat_mode:=True` is enabled, the node subscribes to the `/llm_request_handler` Topic to receive the text entered by the user:

```
# Subscribe to the prompt sent from the text interaction client
self.subscription = self.create_subscription(
    String,
    '/llm_request_handler',
    self.llm_request_handler_callback,
    10
)
```

After receiving the text, it invokes the decision-layer Prompt template, and the cloud/local large model outputs a standardized JSON action queue, for example:

```
# Forward and backward
[
  {"action": "set_cmdvel", "params": {"linear_x": 0.3, "angular_z": 0.0, "time": 3.3}},
  {"action": "set_cmdvel", "params": {"linear_x": -0.08, "angular_z": 0.0, "time": 6.25}}
  {"action": "finish_dialogue", "params": {"text": "Reporting to my master, I have completed
all actions as requested!"}}
]

# Turning template
[
  {"action": "move_left", "params": {"angle": 90, "angular_speed": 0.5}},
  {"action": "move_right", "params": {"angle": 30, "angular_speed": 0.5}}
]
```

4.2 Underlying Action Executor (action_service_*.py)

After the server receives the action goals, it executes them one by one in `CustomActionServer`:

- **Chassis movement**: `set_cmdvel(linear_x, angular_z, time)` continuously broadcasts velocity to `/cmd_vel` and brakes with precise timing;
- **Anthropomorphic performance**: `dance()` drives the multi-DOF servos of the robotic arm to sway and makes fine adjustments to the chassis, displaying agile body language;
- **Status feedback**: reports execution progress through `interfaces.action.Rot`, and releases the lock after the task is completed to enter the next interaction cycle.